

Byte Ordering Revisited

■ Idea

- Short/long/quad words stored in memory as 2/4/8 consecutive bytes
- Which byte is most (least) significant?
- Can cause problems when exchanging binary data between machines

■ Big Endian

- Most significant byte has lowest address
- Sparc

■ Little Endian

- Least significant byte has lowest address
- Intel x86, ARM Android and IOS

■ Bi Endian

- Can be configured either way
- ARM

Byte Ordering Example

```
union {  
    unsigned char c[8];  
    unsigned short s[4];  
    unsigned int i[2];  
    unsigned long l[1];  
} dw;
```

32-bit

c[0]	c[1]	c[2]	c[3]	c[4]	c[5]	c[6]	c[7]
s[0]		s[1]		s[2]		s[3]	
i[0]				i[1]			
l[0]							

64-bit

c[0]	c[1]	c[2]	c[3]	c[4]	c[5]	c[6]	c[7]
s[0]		s[1]		s[2]		s[3]	
i[0]				i[1]			
l[0]							

Byte Ordering Example (Cont).

```
int j;
for (j = 0; j < 8; j++)
    dw.c[j] = 0xf0 + j;

printf("Characters 0-7 ==
[0x%x,0x%x,0x%x,0x%x,0x%x,0x%x,0x%x,0x%x]\n",
    dw.c[0], dw.c[1], dw.c[2], dw.c[3],
    dw.c[4], dw.c[5], dw.c[6], dw.c[7]);

printf("Shorts 0-3 == [0x%x,0x%x,0x%x,0x%x]\n",
    dw.s[0], dw.s[1], dw.s[2], dw.s[3]);

printf("Ints 0-1 == [0x%x,0x%x]\n",
    dw.i[0], dw.i[1]);

printf("Long 0 == [0x%lx]\n",
    dw.l[0]);
```

Byte Ordering on IA32

Little Endian

f0	f1	f2	f3	f4	f5	f6	f7
c[0]	c[1]	c[2]	c[3]	c[4]	c[5]	c[6]	c[7]
s[0]		s[1]		s[2]		s[3]	
i[0]				i[1]			
l[0]							

LSB
MSB
LSB
MSB

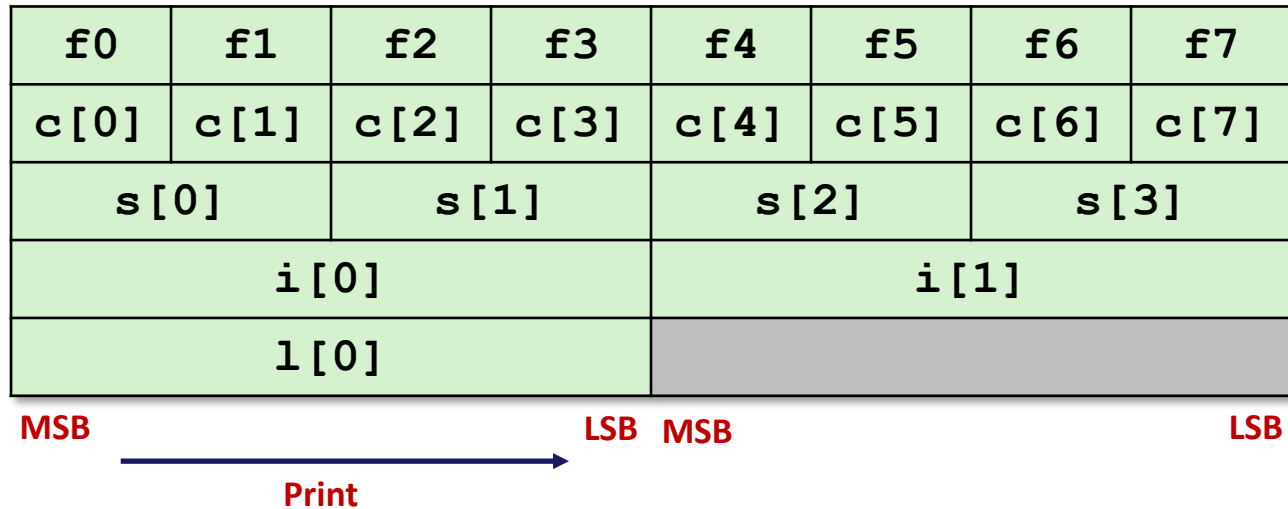
←
Print

Output:

Characters	0-7	==	[0xf0,0xf1,0xf2,0xf3,0xf4,0xf5,0xf6,0xf7]
Shorts	0-3	==	[0xf1f0,0xf3f2,0xf5f4,0xf7f6]
Ints	0-1	==	[0xf3f2f1f0,0xf7f6f5f4]
Long	0	==	[0xf3f2f1f0]

Byte Ordering on Sun

Big Endian



Output on Sun:

Characters 0-7 == [0xf0,0xf1,0xf2,0xf3,0xf4,0xf5,0xf6,0xf7]
Shorts 0-3 == [0xf0f1,0xf2f3,0xf4f5,0xf6f7]
Ints 0-1 == [0xf0f1f2f3,0xf4f5f6f7]
Long 0 == [0xf0f1f2f3]

Byte Ordering on x86-64

Little Endian

f0	f1	f2	f3	f4	f5	f6	f7
c[0]	c[1]	c[2]	c[3]	c[4]	c[5]	c[6]	c[7]
s[0]		s[1]		s[2]		s[3]	
i[0]				i[1]			
l[0]							

LSB

MSB

Print

Output on x86-64:

Characters 0-7 == [0xf0,0xf1,0xf2,0xf3,0xf4,0xf5,0xf6,0xf7]

```
Shorts    0-3 == [0xf1f0,0xf3f2,0xf5f4,0xf7f6]
```

```
Ints      0-1 == [0xf3f2f1f0,0xf7f6f5f4]
```

```
Long      0      ==  [0xf7f6f5f4f3f2f1f0]
```

Summary

■ Arrays in C

- Contiguous allocation of memory
- Aligned to satisfy every element's alignment requirement
- Pointer to first element
- No bounds checking

■ Structures

- Allocate bytes in order declared
- Pad in middle and at end to satisfy alignment

■ Unions

- Overlay declarations
- Way to circumvent type system

IA32 Linux Memory Layout

■ Stack

- Runtime stack (8MB limit)
- E. g., local variables

■ Heap

- Dynamically allocated storage
- When call `malloc()`, `calloc()`, `new()`

■ Data

- Statically allocated data
- E.g., global vars, `static` vars, strings

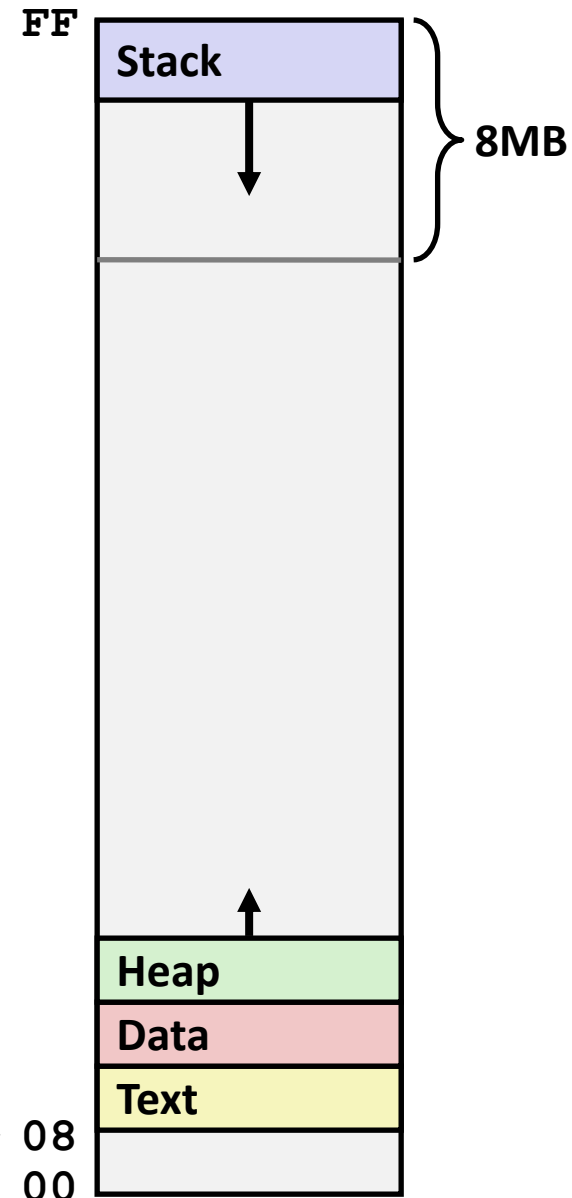
■ Text

- Executable machine instructions
- Read-only

Upper 2 hex digits
= 8 bits of address

08
00

not drawn to scale



Memory Allocation Example

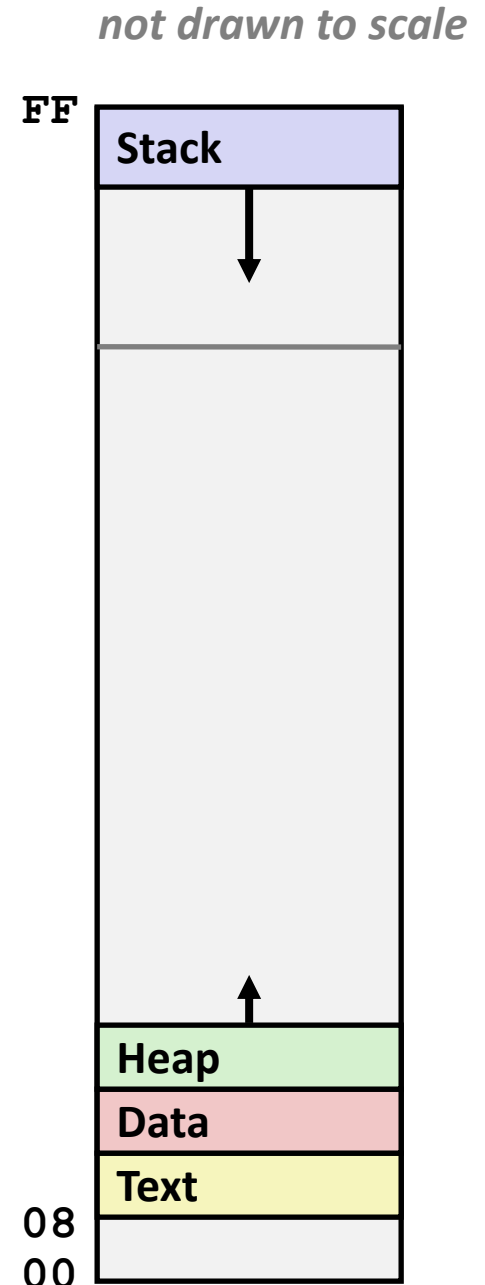
```
char big_array[1<<24]; /* 16 MB */
char huge_array[1<<28]; /* 256 MB */

int beyond;
char *p1, *p2, *p3, *p4;

int useless() { return 0; }

int main()
{
    p1 = malloc(1 <<28); /* 256 MB */
    p2 = malloc(1 << 8); /* 256 B */
    p3 = malloc(1 <<28); /* 256 MB */
    p4 = malloc(1 << 8); /* 256 B */
    /* Some print statements ... */
}
```

Where does everything go?



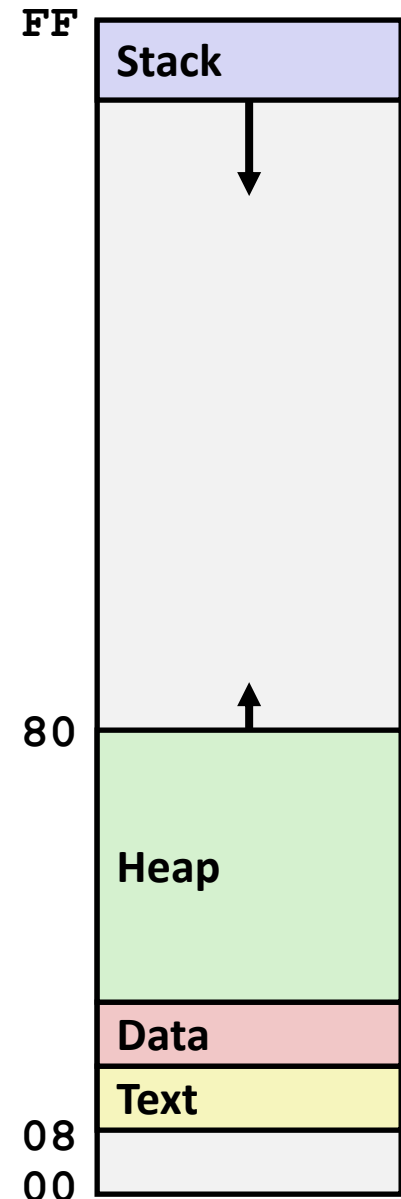
IA32 Example Addresses

address range $\sim 2^{32}$

<code>\$esp</code>	<code>0xffffbcd0</code>
<code>p3</code>	<code>0x65586008</code>
<code>p1</code>	<code>0x55585008</code>
<code>p4</code>	<code>0x1904a110</code>
<code>p2</code>	<code>0x1904a008</code>
<code>&p2</code>	<code>0x18049760</code>
<code>&beyond</code>	<code>0x08049744</code>
<code>big_array</code>	<code>0x18049780</code>
<code>huge_array</code>	<code>0x08049760</code>
<code>main()</code>	<code>0x080483c6</code>
<code>useless()</code>	<code>0x08049744</code>
<code>final malloc()</code>	<code>0x006be166</code>

`malloc()` is dynamically linked
address determined at runtime

not drawn to scale



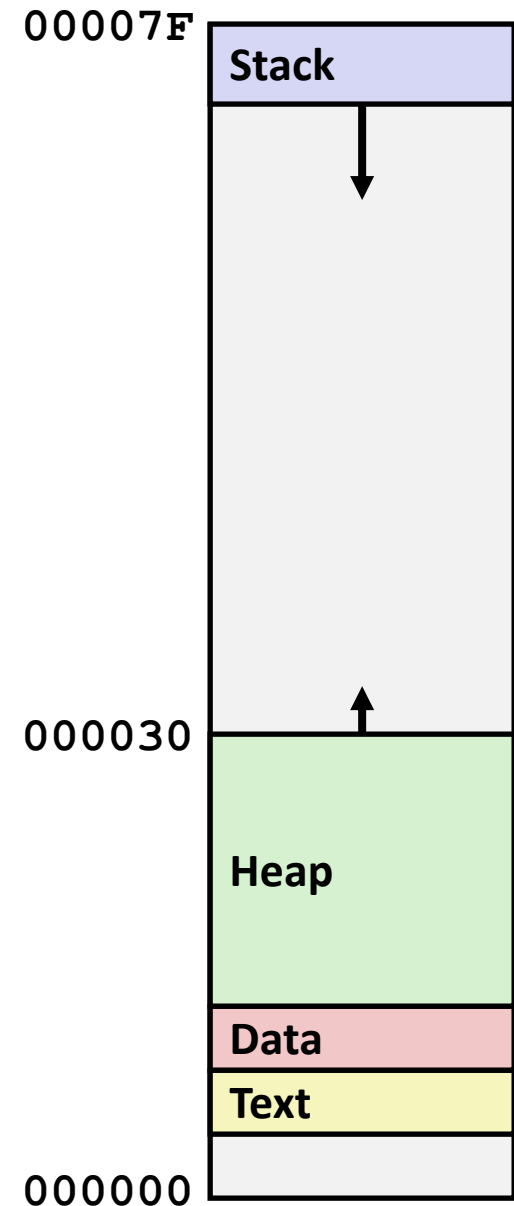
x86-64 Example Addresses

address range $\sim 2^{47}$

<code>\$rsp</code>	<code>0x00007ffffff8d1f8</code>
<code>p3</code>	<code>0x00002aaabaadd010</code>
<code>p1</code>	<code>0x00002aaaaadc010</code>
<code>p4</code>	<code>0x0000000011501120</code>
<code>p2</code>	<code>0x0000000011501010</code>
<code>&p2</code>	<code>0x0000000010500a60</code>
<code>&beyond</code>	<code>0x0000000000500a44</code>
<code>big_array</code>	<code>0x0000000010500a80</code>
<code>huge_array</code>	<code>0x0000000000500a50</code>
<code>main()</code>	<code>0x0000000000400510</code>
<code>useless()</code>	<code>0x0000000000400500</code>
<code>final malloc()</code>	<code>0x000000386ae6a170</code>

`malloc()` is dynamically linked
address determined at runtime

not drawn to scale



Internet Worm and IM War

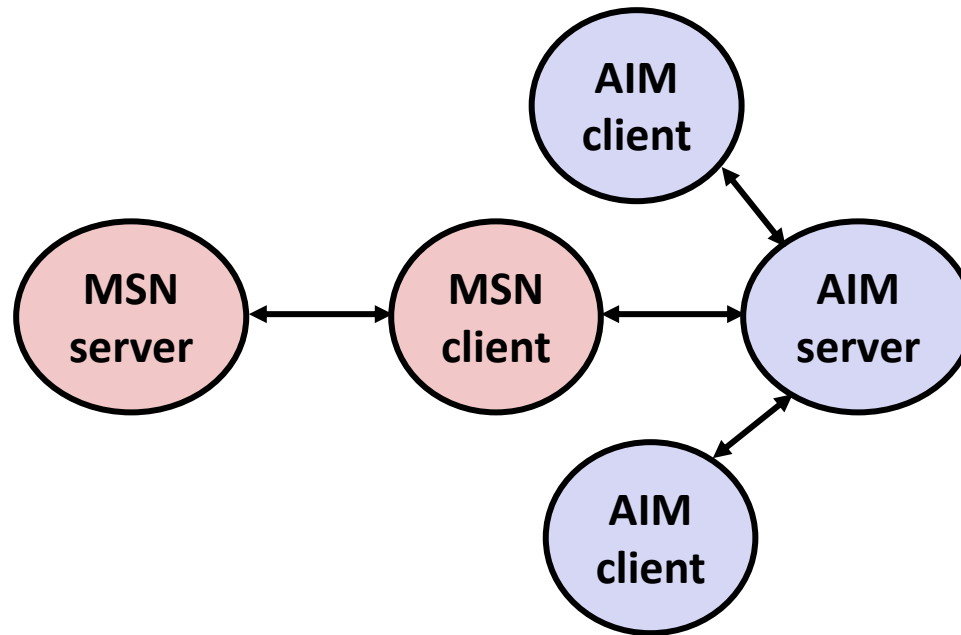
■ November, 1988

- Internet Worm attacks thousands of Internet hosts.
- How did it happen?

Internet Worm and IM War

■ July, 1999

- Microsoft launches MSN Messenger (instant messaging system).
- Messenger clients can access popular AOL Instant Messaging Service (AIM) servers



Internet Worm and IM War (cont.)

■ August 1999

- Mysteriously, Messenger clients can no longer access AIM servers.
- Microsoft and AOL begin the IM war:
 - AOL changes server to disallow Messenger clients
 - Microsoft makes changes to clients to defeat AOL changes.
 - At least 13 such skirmishes.
- How did it happen?

Internet Worm and IM War (cont.)

- The Internet Worm and AOL/Microsoft War were both based on *stack buffer overflow* exploits!
 - many library functions do not check argument sizes.
 - allows target buffers to overflow.

String Library Code

■ Implementation of Unix function `gets()`

```
/* Get string from stdin */
char *gets(char *dest)
{
    int c = getchar();
    char *p = dest;
    while (c != EOF && c != '\n') {
        *p++ = c;
        c = getchar();
    }
    *p = '\0';
    return dest;
}
```

- No way to specify limit on number of characters to read
- **Similar problems with other library functions**
 - `strcpy`, `strcat`: Copy strings of arbitrary length
 - `scanf`, `fscanf`, `sscanf`, when given `%s` conversion specification

Vulnerable Buffer Code

```
/* Echo Line */  
void echo()  
{  
    char buf[4];  /* Way too small! */  
    gets(buf);  
    puts(buf);  
}
```

```
void call_echo() {  
    echo();  
}
```

```
unix>./bufdemo  
Type a string:1234567  
1234567
```

```
unix>./bufdemo  
Type a string:12345678  
Segmentation Fault
```

```
unix>./bufdemo  
Type a string:123456789ABC  
Segmentation Fault
```

Buffer Overflow Disassembly

echo:

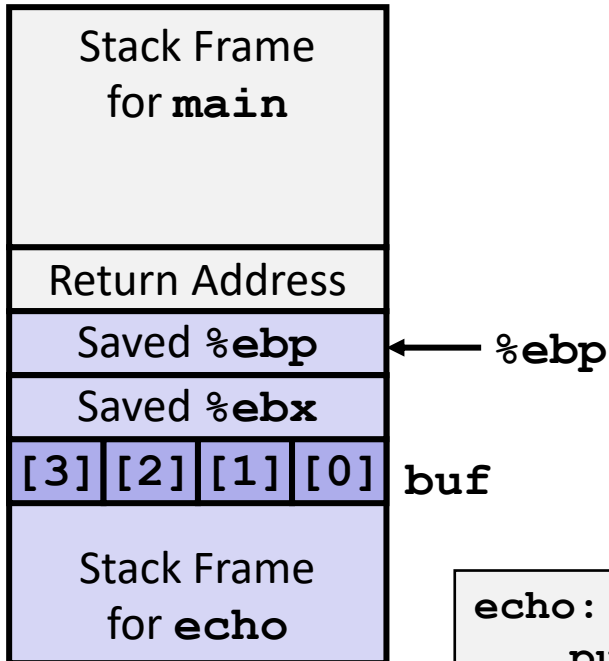
80485c5:	55	push	%ebp
80485c6:	89 e5	mov	%esp, %ebp
80485c8:	53	push	%ebx
80485c9:	83 ec 14	sub	\$0x14, %esp
80485cc:	8d 5d f8	lea	0xfffffffff8(%ebp), %ebx
80485cf:	89 1c 24	mov	%ebx, (%esp)
80485d2:	e8 9e ff ff ff	call	8048575 <gets>
80485d7:	89 1c 24	mov	%ebx, (%esp)
80485da:	e8 05 fe ff ff	call	80483e4 <puts@plt>
80485df:	83 c4 14	add	\$0x14, %esp
80485e2:	5b	pop	%ebx
80485e3:	5d	pop	%ebp
80485e4:	c3	ret	

call_echo:

80485eb:	e8 d5 ff ff ff	call	80485c5 <echo>
80485f0:	c9	leave	
80485f1:	c3	ret	

Buffer Overflow Stack

Before call to gets



```
/* Echo Line */
void echo()
{
    char buf[4]; /* Way too small! */
    gets(buf);
    puts(buf);
}
```

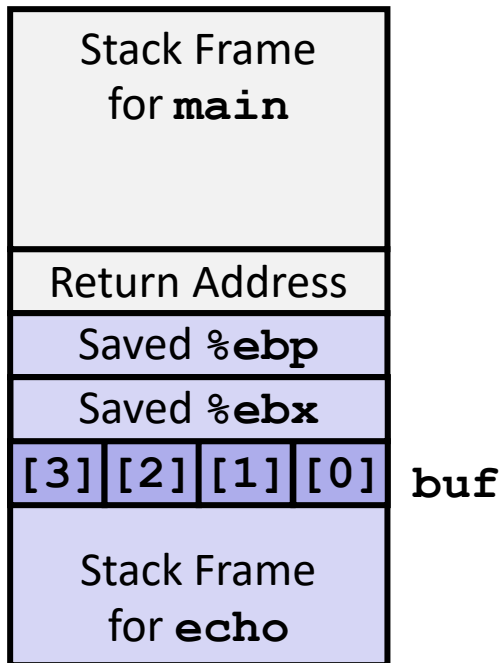
```
echo:
    pushl %ebp                # Save %ebp on stack
    movl  %esp, %ebp
    pushl %ebx                # Save %ebx
    subl  $20, %esp           # Allocate stack space
    leal  -8(%ebp), %ebx      # Compute buf as %ebp-8
    movl  %ebx, (%esp)        # Push buf on stack
    call  gets                # Call gets
    . . .
```

Buffer Overflow Stack Example

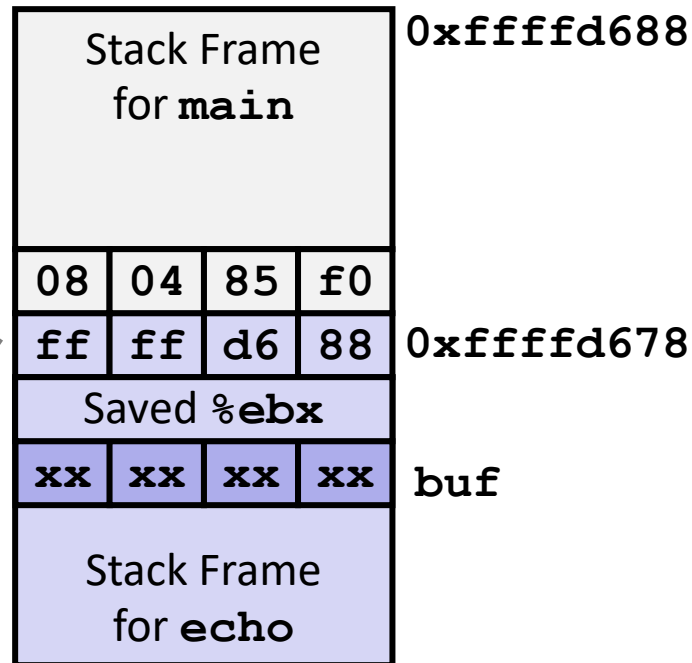
```

unix> gdb bufdemo
(gdb) break echo
Breakpoint 1 at 0x80485c9
(gdb) run
Breakpoint 1, 0x80485c9 in echo ()
(gdb) print /x $ebp
$1 = 0xffffd678
(gdb) print /x *(unsigned *)$ebp
$2 = 0xffffd688
(gdb) print /x *((unsigned *)$ebp + 1)
$3 = 0x80485f0
  
```

Before call to gets



Before call to gets



```

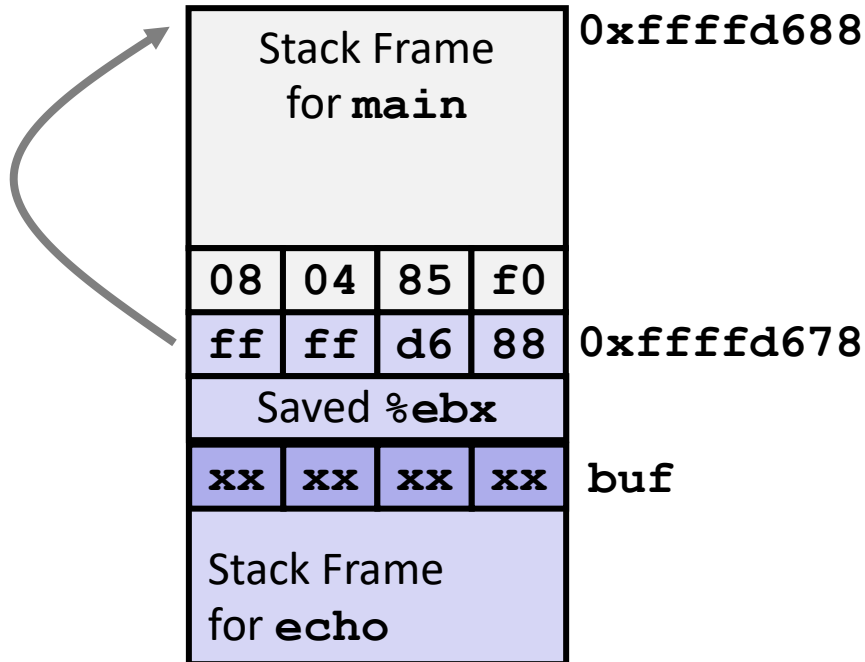
80485eb:  e8 d5 ff ff ff
80485f0:  c9
  
```

```

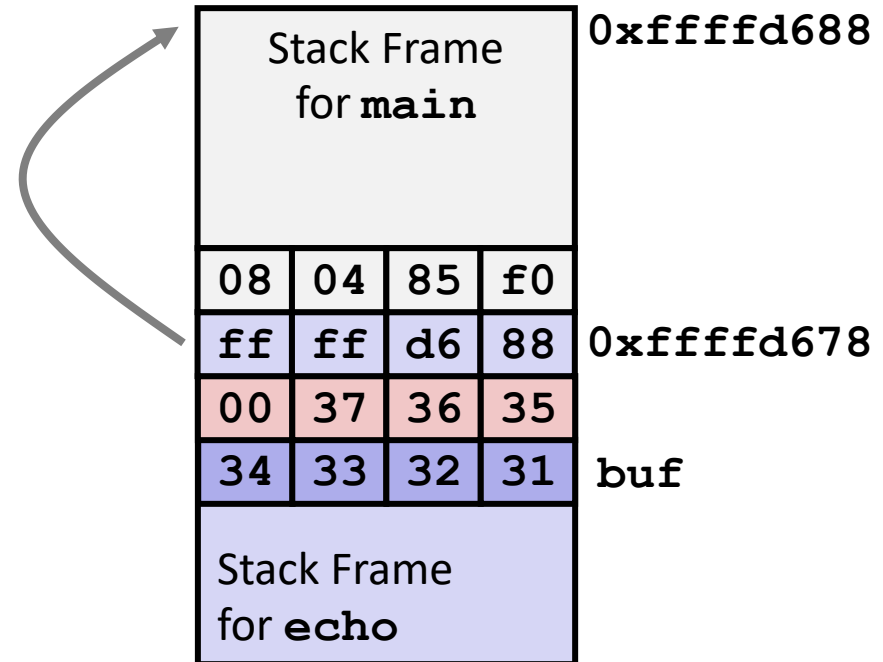
call 80485c5 <echo>
leave
  
```

Buffer Overflow Example #1

Before call to gets



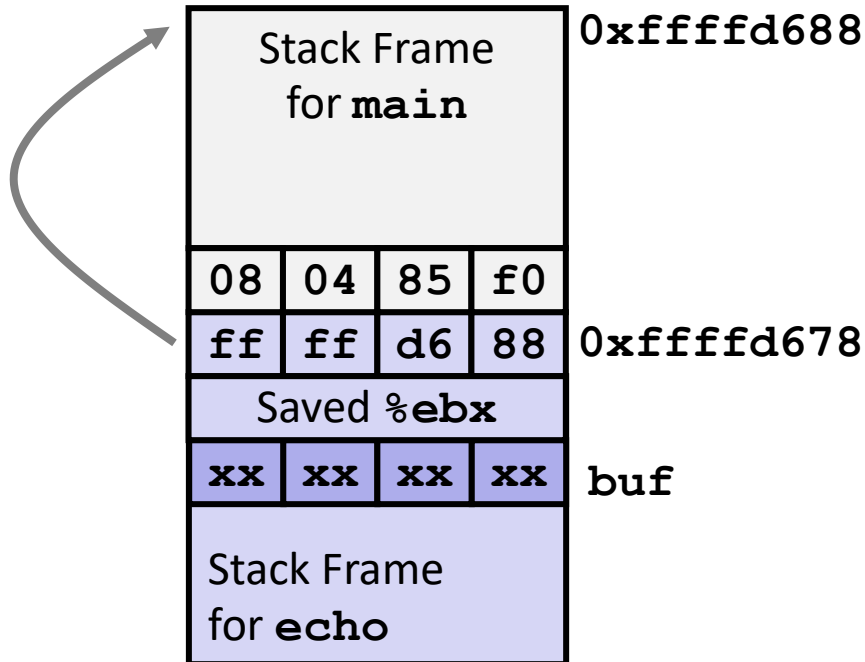
Input 1234567



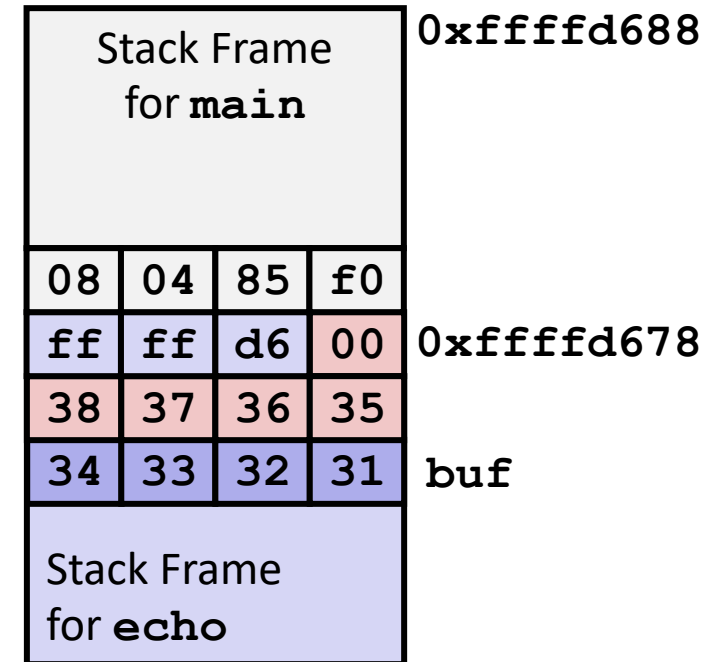
**Overflow buf, and corrupt %ebx,
but no problem**

Buffer Overflow Example #2

Before call to gets



Input 12345678



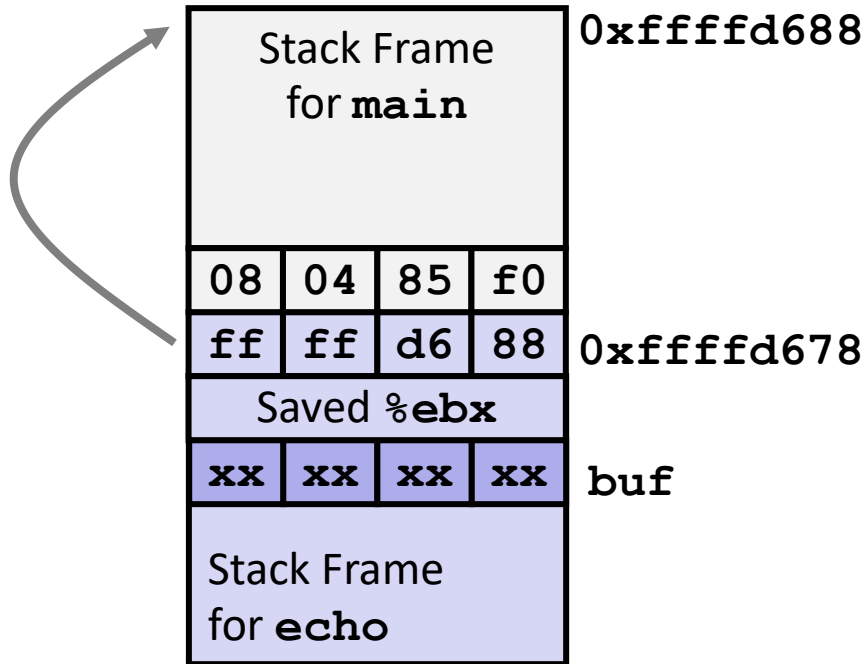
Base pointer corrupted

```

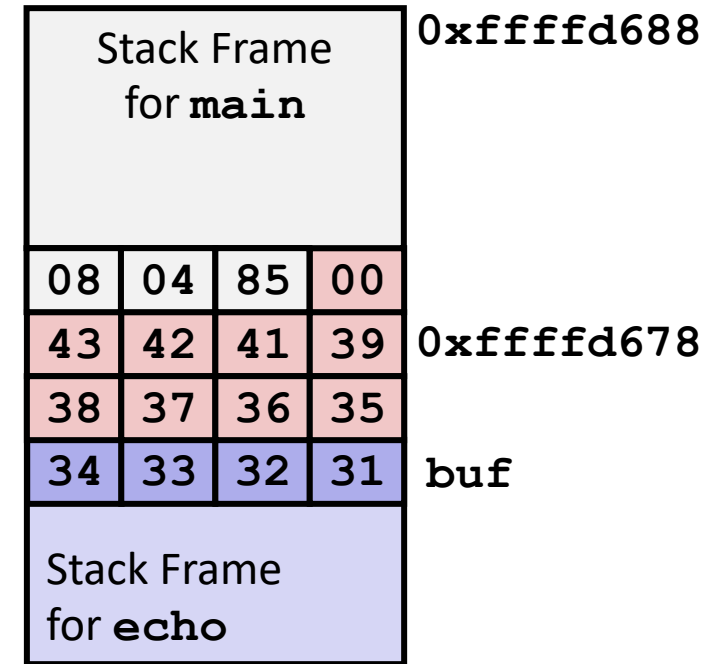
. . .
80485eb:  e8 d5 ff ff ff  call    80485c5 <echo>
80485f0:  c9                leave   # Set %ebp to corrupted value
80485f1:  c3                ret
    
```

Buffer Overflow Example #3

Before call to gets



Input 123456789ABC

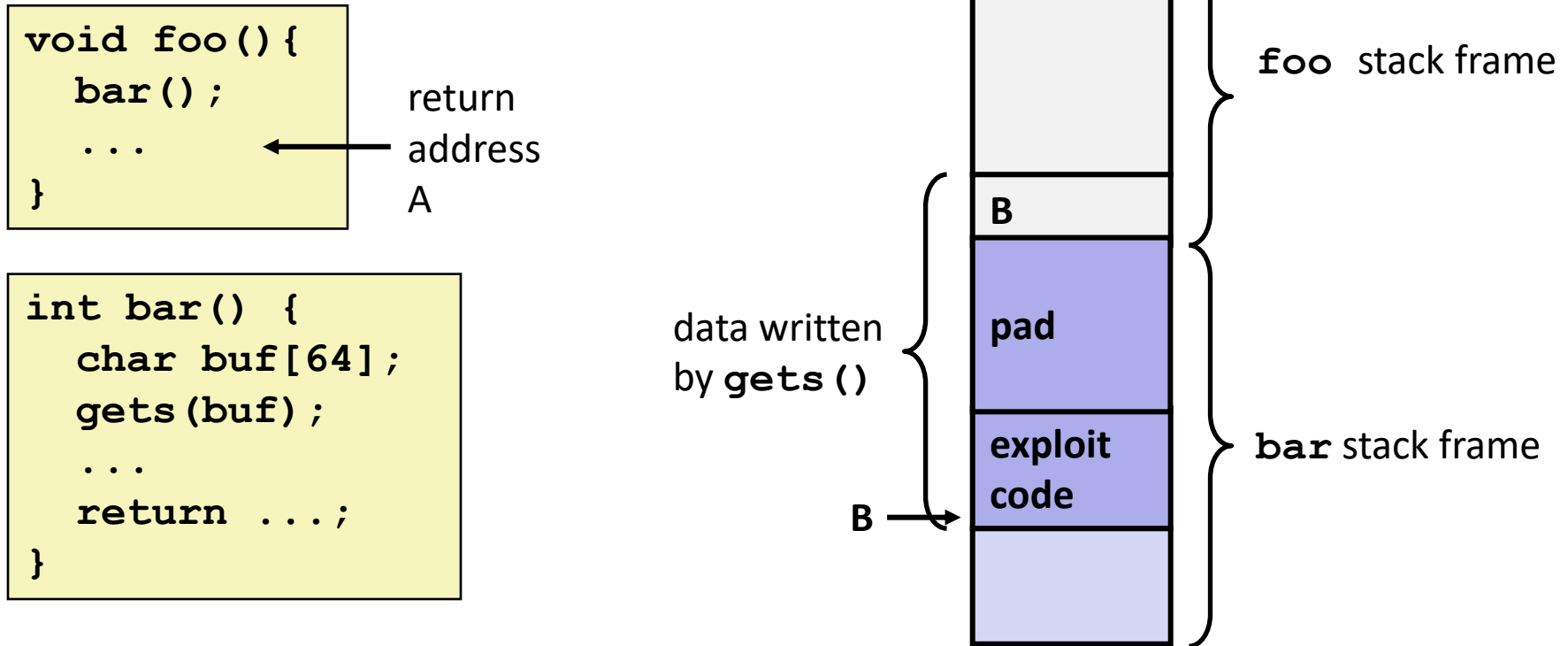


Return address corrupted

```

80485eb:  e8 d5 ff ff ff  call 80485c5 <echo>
80485f0:  c9                leave # Desired return point
    
```

Malicious Use of Buffer Overflow



- Input string contains byte representation of executable code
- Overwrite return address A with address of buffer B
- When `bar()` executes `ret`, will jump to exploit code

Exploits Based on Buffer Overflows

- *Buffer overflow bugs allow remote machines to execute arbitrary code on victim machines*
- **Internet worm**
 - Early versions of the finger server (fingerd) used `gets ()` to read the argument sent by the client:
 - `finger droh@cs.cmu.edu`
 - Worm attacked fingerd server by sending phony argument:
 - `finger "exploit-code padding new-return-address"`
 - exploit code: executed a root shell on the victim machine with a direct TCP connection to the attacker.

Exploits Based on Buffer Overflows

- *Buffer overflow bugs allow remote machines to execute arbitrary code on victim machines*
- **IM War**
 - AOL exploited existing buffer overflow bug in AIM clients
 - exploit code: returned 4-byte signature (the bytes at some location in the AIM client) to server.
 - When Microsoft changed code to match signature, AOL changed signature location.

Date: Wed, 11 Aug 1999 11:30:57 -0700 (PDT)
From: Phil Bucking <philbucking@yahoo.com>
Subject: AOL exploiting buffer overrun bug in their own software!
To: rms@pharlap.com

Mr. Smith,

I am writing you because I have discovered something that I think you might find interesting because you are an Internet security expert with experience in this area. I have also tried to contact AOL but received no response.

I am a developer who has been working on a revolutionary new instant messaging client that should be released later this year.

...

It appears that the AIM client has a buffer overrun bug. By itself this might not be the end of the world, as MS surely has had its share. But AOL is now *exploiting their own buffer overrun bug* to help in its efforts to block MS Instant Messenger.

....

Since you have significant credibility with the press I hope that you can use this information to help inform people that behind AOL's friendly exterior they are nefariously compromising peoples' security.

Sincerely,
Phil Bucking
Founder, Bucking Consulting
philbucking@yahoo.com

***It was later determined that this
email originated from within
Microsoft!***

Avoiding Overflow Vulnerability

```
/* Echo Line */  
void echo()  
{  
    char buf[4]; /* Way too small! */  
    fgets(buf, 4, stdin);  
    puts(buf);  
}
```

- **Use library routines that limit string lengths**
 - **fgets** instead of **gets**
 - **strncpy** instead of **strcpy**
 - Don't use **scanf** with **%s** conversion specification
 - Use **fgets** to read the string
 - Or use **%ns** where **n** is a suitable integer

System-Level Protections

■ Randomized stack offsets

- At start of program, allocate random amount of space on stack
- Makes it difficult for hacker to predict beginning of inserted code

■ Nonexecutable code segments

- In traditional x86, can mark region of memory as either “read-only” or “writeable”
 - Can execute anything readable
- X86-64 added explicit “execute” permission

```
unix> gdb bufdemo
(gdb) break echo

(gdb) run
(gdb) print /x $ebp
$1 = 0xfffffc638

(gdb) run
(gdb) print /x $ebp
$2 = 0xffffbb08

(gdb) run
(gdb) print /x $ebp
$3 = 0xffffc6a8
```

Stack Canaries

■ Idea

- Place special value (“canary”) on stack just beyond buffer
- Check for corruption before exiting function

■ GCC Implementation

- `-fstack-protector`
- `-fstack-protector-all`

```
unix>./bufdemo-protected  
Type a string:1234  
1234
```

```
unix>./bufdemo-protected  
Type a string:12345  
*** stack smashing detected ***
```

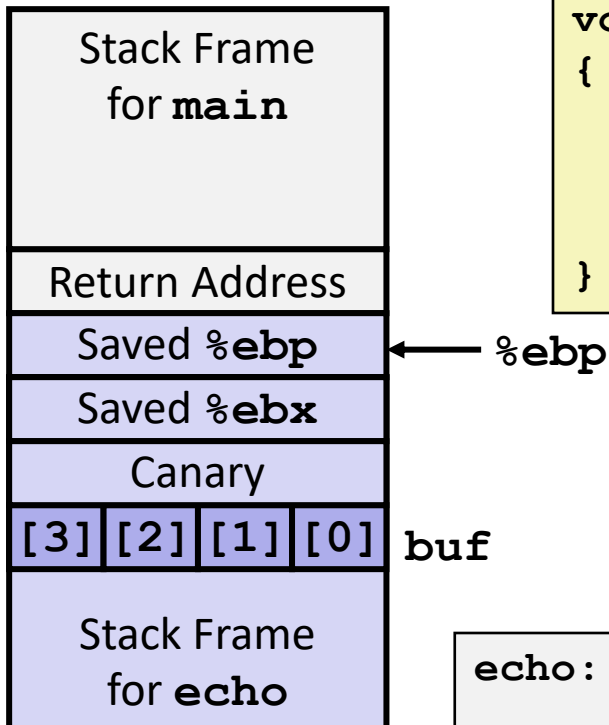
Protected Buffer Disassembly

echo:

804864d:	55	push	%ebp
804864e:	89 e5	mov	%esp, %ebp
8048650:	53	push	%ebx
8048651:	83 ec 14	sub	\$0x14, %esp
8048654:	65 a1 14 00 00 00	mov	%gs:0x14, %eax
804865a:	89 45 f8	mov	%eax, 0xffffffff8(%ebp)
804865d:	31 c0	xor	%eax, %eax
804865f:	8d 5d f4	lea	0xfffffffff4(%ebp), %ebx
8048662:	89 1c 24	mov	%ebx, (%esp)
8048665:	e8 77 ff ff ff	call	80485e1 <gets>
804866a:	89 1c 24	mov	%ebx, (%esp)
804866d:	e8 ca fd ff ff	call	804843c <puts@plt>
8048672:	8b 45 f8	mov	0xfffffffff8(%ebp), %eax
8048675:	65 33 05 14 00 00 00	xor	%gs:0x14, %eax
804867c:	74 05	je	8048683 <echo+0x36>
804867e:	e8 a9 fd ff ff	call	804842c <FAIL>
8048683:	83 c4 14	add	\$0x14, %esp
8048686:	5b	pop	%ebx
8048687:	5d	pop	%ebp
8048688:	c3	ret	

Setting Up Canary

Before call to gets

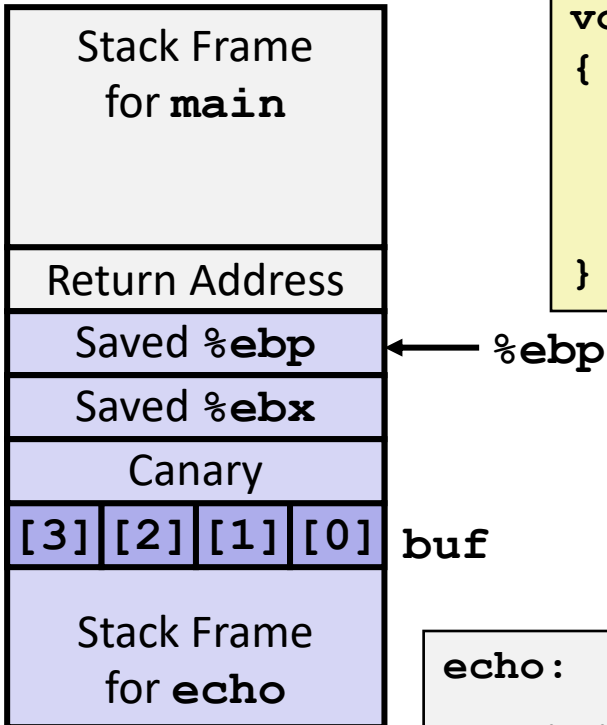


```
/* Echo Line */
void echo()
{
    char buf[4]; /* Way too small! */
    gets(buf);
    puts(buf);
}
```

```
echo:
    . . .
    movl    %gs:20, %eax    # Get canary
    movl    %eax, -8(%ebp)  # Put on stack
    xorl    %eax, %eax      # Erase canary
    . . .
```


Checking Canary

Before call to gets

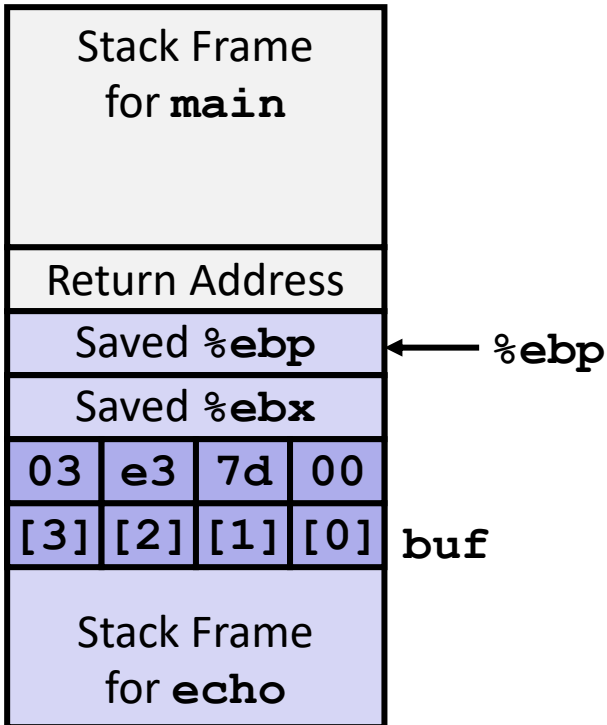


```
/* Echo Line */
void echo()
{
    char buf[4]; /* Way too small! */
    gets(buf);
    puts(buf);
}
```

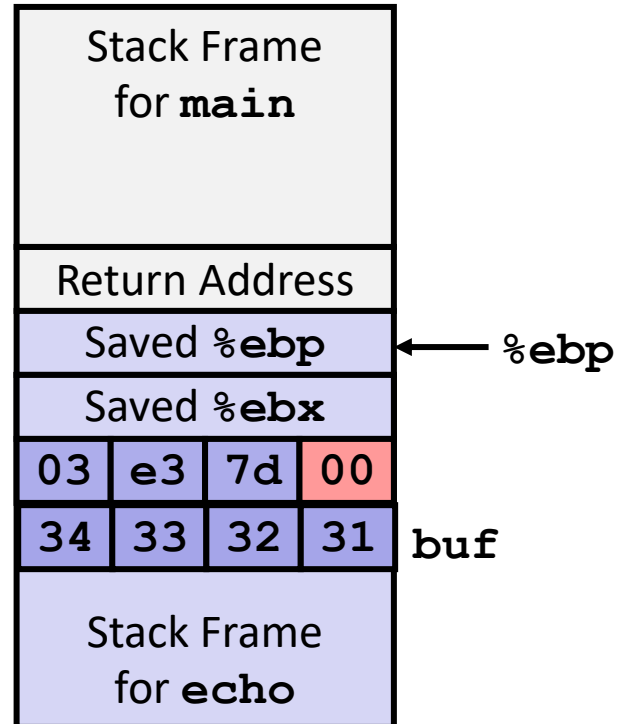
```
echo:
    . . .
    movl    -8(%ebp), %eax    # Retrieve from stack
    xorl    %gs:20, %eax     # Compare with Canary
    je      .L24              # Same: skip ahead
    call    __stack_chk_fail # ERROR
.L24:
    . . .
```

Canary Example

Before call to gets



Input 1234



```
(gdb) break echo
(gdb) run
(gdb) stepi 3
(gdb) print /x *((unsigned *) $ebp - 2)
$1 = 0x3e37d00
```

Benign corruption!
(allows programmers to make
silent off-by-one errors)

Worms and Viruses

- **Worm: A program that**

- Can run by itself
- Can propagate a fully working version of itself to other computers

- **Virus: Code that**

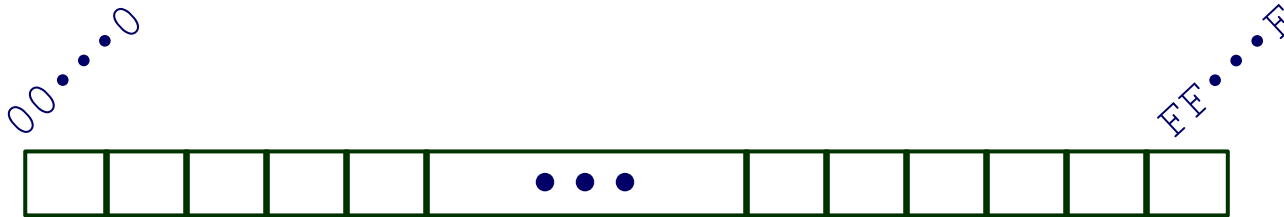
- Add itself to other programs
- Cannot run independently

- **Both are (usually) designed to spread among computers and to wreak havoc**

Summary

- **Structures**
 - Alignment
- **Unions**
- **Memory Layout**
- **Buffer Overflow**
 - Vulnerability
 - Protection

Byte-Oriented Memory Organization



■ Programs refer to data by address

- Conceptually, envision it as a very large array of bytes
 - In reality, it's not, but can think of it that way
- An address is like an index into that array
 - and, a pointer variable stores an address

■ Note: system provides private address spaces to each “process”

- Think of a process as a program being executed
- So, a program can clobber its own data, but not that of others

Simple Memory Addressing Modes

■ Normal (R) Mem[Reg[R]]

- Register R specifies memory address
- Aha! Pointer dereferencing in C

```
movl (%ecx) , %eax
```

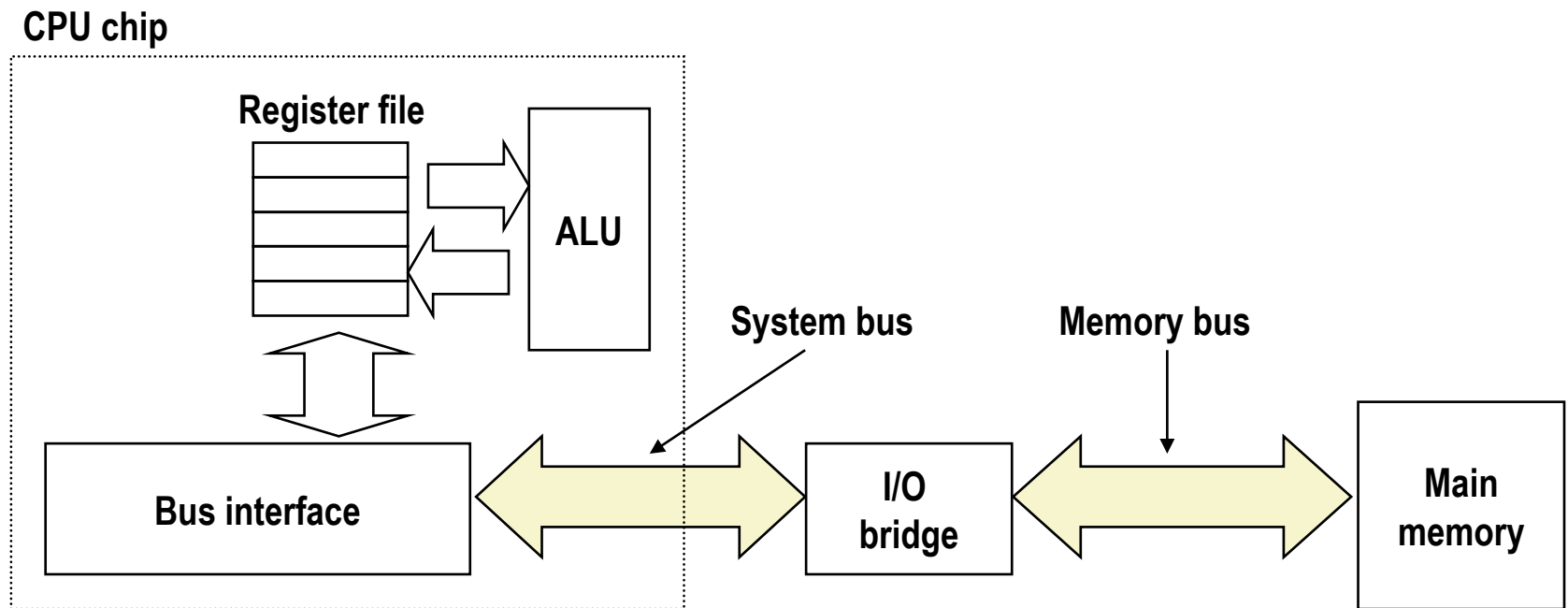
■ Displacement D(R) Mem[Reg[R]+D]

- Register R specifies start of memory region
- Constant displacement D specifies offset

```
movl 8(%ebp) , %edx
```

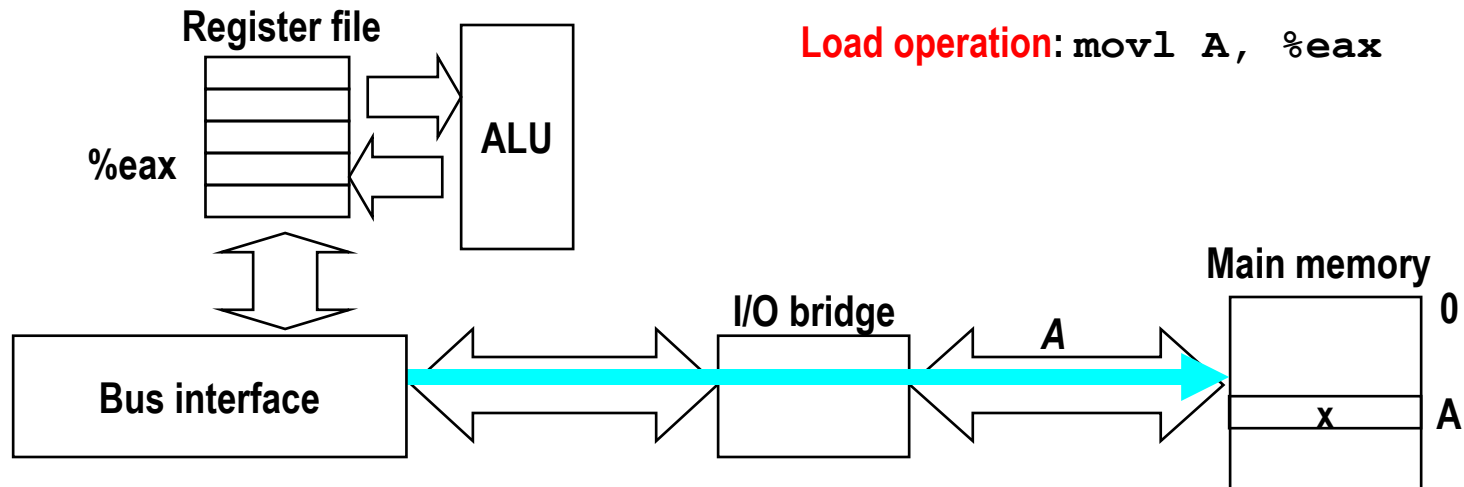
Traditional Bus Structure Connecting CPU and Memory

- A **bus** is a collection of parallel wires that carry address, data, and control signals.
- Buses are typically shared by multiple devices.



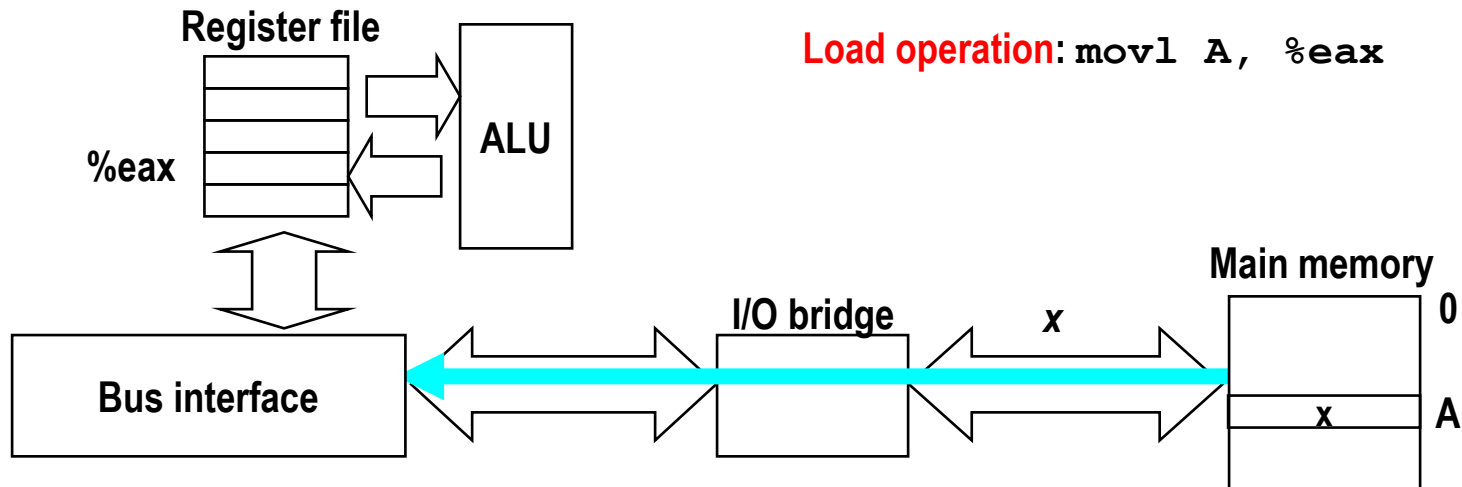
Memory Read Transaction (1)

- CPU places address A on the memory bus.



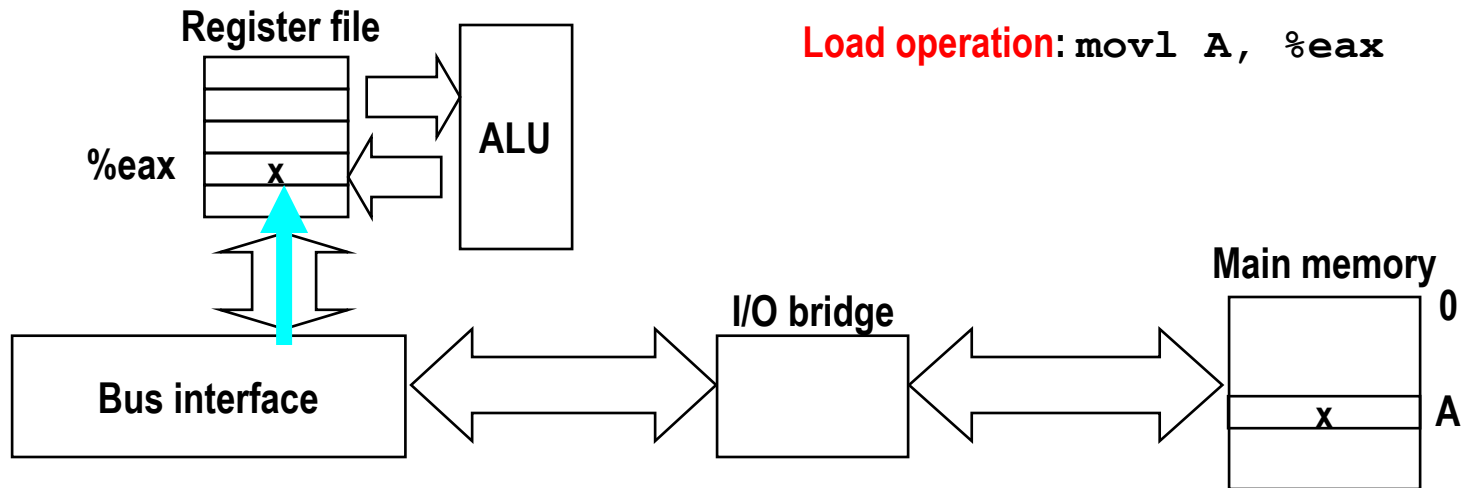
Memory Read Transaction (2)

- Main memory reads *A* from the memory bus, retrieves word *x*, and places it on the bus.



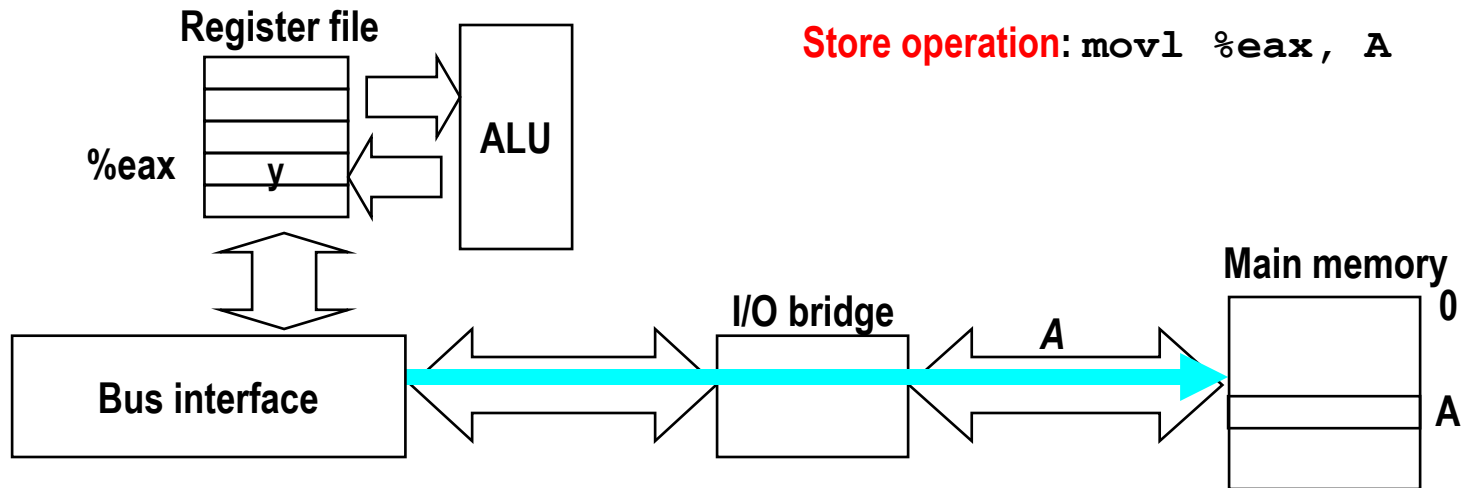
Memory Read Transaction (3)

- CPU read word x from the bus and copies it into register `%eax`.



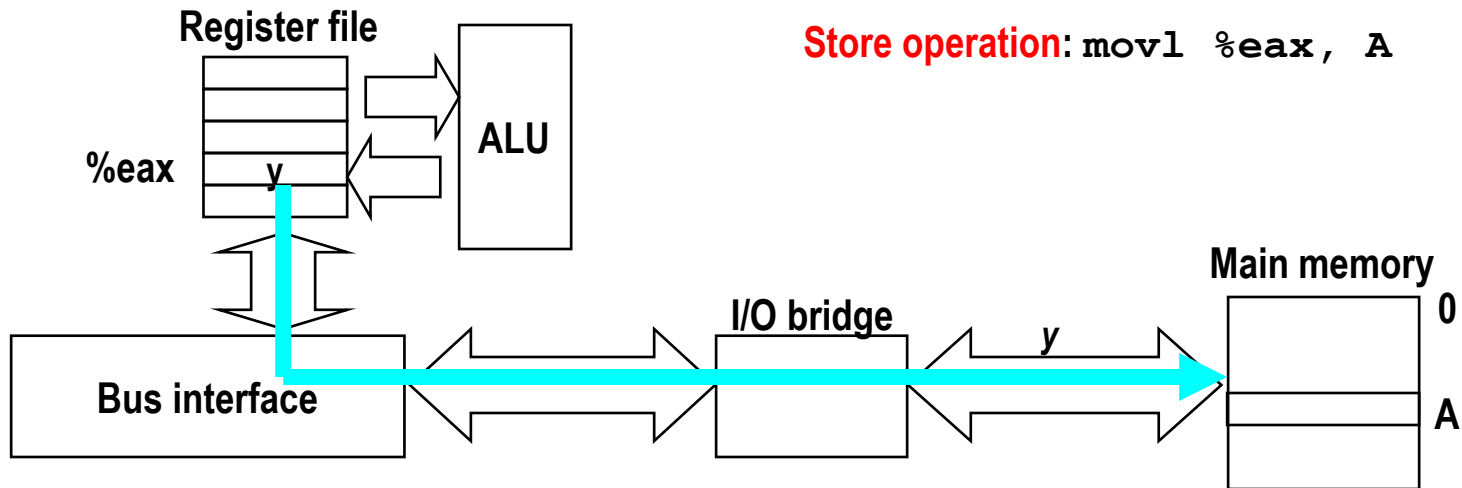
Memory Write Transaction (1)

- CPU places address A on bus. Main memory reads it and waits for the corresponding data word to arrive.



Memory Write Transaction (2)

- CPU places data word y on the bus.



Memory Write Transaction (3)

- Main memory reads data word *y* from the bus and stores it at address *A*.

